



MAHALAKSHMI TECH CAMPUS INSTITUTE OF ENGINEERS

(A GROUP OF MAHALAKSHMI INSTITUTIONS)
Approved by AICTE & Affiliated to Anna University, Chennai.



PROJECT REPORT

SERVICE-NOW ADMINISTRATOR

AUTO TICKET CLASSIFICATION USING FLOW DESIGNER

ACADEMIC YEAR 2026-2027 (ODD SEMESTER)

NAME :

REGISTER NUMBER :

DEGREE & SEMESTER :

YEAR & BRANCH :

REGULATION : 2021



MAHALAKSHMI TECH CAMPUS INSTITUTE OF ENGINEERS

(A GROUP OF MAHALAKSHMI INSTITUTIONS)
Approved by AICTE & Affiliated to Anna University, Chennai.

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

BONAFIDE CERTIFICATE

This is to certify that this “Auto Ticket Classification Using Flow Designer” (Service Now Administrator) Project work done by Mr/Ms for the course (Degree/Department) during (Year/Semester) of academic year 2026–2027 is Bonafide.

Staff in-charge

Head of the Department

This Project Report is submitted for Degree/Semester Practical Examination of Anna University conducted on

INTERNAL EXAMINER

EXTERNAL EXAMINER

PROJECT ABSTRACT

The project "Auto Ticket Classification Using Flow Designer" presents the design of a rule-based workflow that classifies IT support records consistently in ServiceNow. Service desk agents often have to read each new record and choose a category by hand. This work is repetitive, and similar problems may be classified differently by different people. Requests raised through the Service Catalog have a related gap: the catalog item that was ordered describes the request, but this information is not always turned into a consistent classification.

The proposed solution uses ServiceNow Flow Designer to evaluate documented conditions and decision logic, not machine learning or artificial intelligence. Two separate flows are designed. The Incident flow evaluates the short description and other structured fields through independent rule checks. The Request flow classifies Requested Items raised from existing Service Catalog items, using the catalog item as its primary signal and reading catalog variables only when the item alone is not enough. Both flows apply a common decision process that allows a value to be written only when exactly one valid and compatible classification remains. A field-preservation guard then writes only blank fields and never overwrites existing values. Ambiguous, conflicting and unmatched records are left unchanged and sent to a review path. Automatic assignment, approval, rejection, fulfilment and closure of requests are outside the scope of this project.

This report documents the requirements, architecture, workflows, rule register, decision logic, configuration procedure and a planned testing strategy of eighteen test cases. It is a design report: the flows have not yet been configured or executed, no results are claimed, and instance-specific details such as field names, choice values, catalog items and trigger options are marked for confirmation in the target ServiceNow instance.

Keywords: ServiceNow, Flow Designer, Incident Management, Service Catalog, Requested Item, Rule-Based Classification, Workflow Automation, Field Preservation.

TABLE OF CONTENTS

Chapter / Section	Page
1. INTRODUCTION	1
1.1 Purpose of the System	1
1.2 Scope of the Project	2
1.3 Technologies Used	3
1.4 Project Objectives	4
1.5 Target Audience	4
1.6 Benefits of the System	4
2. SYSTEM REQUIREMENTS	5
2.1 Functional Requirements	5
2.2 Non-Functional Requirements	5
2.3 Hardware Requirements	6
2.4 Software and Platform Requirements	7
2.5 Network Requirements	7
2.6 Data Model and Record Design	7
3. CODING: FLOW DESIGN AND CONFIGURATION	9
3.1 Proposed System Architecture	9
3.2 Overall Classification Workflow	10
3.3 Flow Designer Triggers, Conditions and Actions	10
3.4 Rule-Based Incident Classification	12
3.5 Classification of Existing Service Catalog Request Items	13
3.6 Classification Decision Process	15
3.7 Record-Field Handling and Preservation of Existing Values	17
3.8 Implementation Methodology and Configuration Steps	19
3.9 Expected Outputs	20
4. TESTING	21
4.1 Testing Strategy	21
4.2 Test Environment and Preconditions	21
4.3 Planned Test Cases	21
4.4 Detailed Test Case Descriptions	22
4.5 Evaluation Approach and Acceptance Criteria	23
5. TECHNICAL CHALLENGES	24
5.1 Variation in Free-Text Descriptions	24
5.2 Overlapping and Conflicting Rules	24
5.3 Variability of Catalog Items and Variables	24
5.4 Trigger Selection for Existing Catalog Items	24
5.5 Flow Re-triggering and Repeated Updates	24
5.6 Instance Permissions and Unverified Configuration	25
6. DESIGN CHALLENGES AND CONSIDERATIONS	25
6.1 Balancing Automation and Control	25
6.2 Separating Classification from Routing and Fulfilment	25
6.3 Explainability and Maintainability	25
6.4 Design Considerations and Limitations	25
7. FUTURE ENHANCEMENTS	26
7.1 Support-Group Routing	26
7.2 Easier Rule Management	26
7.3 Expanded Catalog Coverage	26
7.4 Reporting and Monitoring	26
7.5 AI-Assisted Suggestions with Human Review	26
7.6 Phased Roadmap	26
8. CONCLUSION	27
8.1 Project Goals and Expected Outcomes	27
8.2 Reflection on Project Learnings	27
8.3 Final Thoughts	27
REFERENCES	28
APPENDIX A: TEST RECORD AND EVIDENCE TEMPLATES	29
APPENDIX B: SERVICENOW CERTIFICATE	30

AUTO TICKET CLASSIFICATION USING FLOW DESIGNER

1. INTRODUCTION

1.1 Purpose of the System

Organisations that use an IT service management (ITSM) platform receive a steady flow of support records. Some of these records report that something has stopped working, while others ask for something new, such as software, equipment or access. In ServiceNow, the first kind is recorded as an **Incident** and the second is usually raised through the **Service Catalog** as a request, which produces a **Requested Item** for the catalog item that was ordered. Before such a record can be reported on, routed or analysed reliably, it should carry a consistent classification.

The purpose of this project is to design a rule-based classification workflow in **ServiceNow Flow Designer** that fills supported classification fields on eligible Incident records and on Requested Items raised from existing Service Catalog items. The workflow evaluates documented conditions against information that is already present on the record. It writes a value only when the evidence clearly supports exactly one outcome, and only into fields that are still blank. Records that are ambiguous, conflicting or unmatched are left unchanged and directed to a review path so that a person can decide.

This report is a **design and documentation report**. It describes the intended solution, the configuration approach and a planned testing strategy. The flows have not yet been built, executed or tested in a ServiceNow instance, and no results are reported. Exact field names, choice values, catalog items and trigger options are marked as subject to confirmation wherever they depend on the target instance.

1.1.1 Background

ServiceNow documentation describes Incident Management as a process for identifying, logging, classifying, prioritising, assigning, escalating, resolving and reporting incidents, and identifies incident categorisation as an important step in that process [5], [6]. Classification affects what happens later: reports grouped by category, queue views used by support teams and any future routing rules all depend on the category values being reliable. When users describe problems in their own words, the same underlying issue can appear in many forms, for example "Wi-Fi keeps dropping", "no internet on my laptop" or "cannot reach the shared drive".

Flow Designer is ServiceNow's low-code process automation builder. A flow is made of a trigger that decides when the flow runs, followed by actions and flow logic that perform the work [1]. In recent ServiceNow releases the documentation presents this capability under the name **Workflow Studio**; this report uses the term Flow Designer, as in the project title, to refer to the same flow-building capability.

1.1.2 Problem Statement

Service desk agents frequently have to read each new record, interpret its description and select a category and subcategory by hand. This work is repetitive, and two agents may classify the same kind of problem differently. Requests raised from catalog items raise a related difficulty: the ordered item usually says what the request is about, but this information is not always turned into a consistent classification. The problem addressed by this project is therefore stated as follows:

How can eligible Incident records and Requested Items from existing catalog items be classified consistently and transparently, using documented rules in Flow Designer, without overwriting values that people have already entered and without guessing when the evidence is unclear?

The problem is deliberately narrower than "automating the service desk". Assigning records to support groups, approving or rejecting requests, fulfilling requests and closing records are separate decisions with their own risks, and they are outside the scope of this project.

1.1.3 Existing System and Its Limitations

In the existing approach, classification depends mainly on the person who creates or first handles the record. The limitations of this approach, and the way the proposed system responds to each, are summarised in Table 1.1.

Table 1.1: Comparison of the existing manual approach with the proposed system

Aspect	Existing manual approach	Proposed rule-based approach
Consistency	Depends on individual interpretation; similar records may receive different categories.	The same documented rules are applied to every eligible record in the same order.
Effort	Agents repeat the same reading and selection work for routine records.	Clear, routine cases can be classified automatically; agents focus on unclear ones.
Existing values	A later edit can accidentally replace a value someone chose earlier.	A field guard writes only blank fields and preserves populated ones.
Unclear records	May be classified on a guess to clear the queue.	Ambiguous, conflicting and unmatched records are left unchanged and sent to review.
Catalog requests	The ordered item is not consistently translated into a classification.	The existing catalog item is used as the primary signal, with variables only where needed.
Transparency	The reason for a choice is rarely recorded.	Each outcome can be traced to a rule ID in the rule register and to flow execution details.

1.1.4 Proposed System

The proposed system consists of two logically separate Flow Designer flows: one for Incident records and one for Requested Items. Each flow reads the relevant inputs, evaluates a set of independent rule checks, resolves the result to one clear outcome or to a review case, and then applies a field-preservation guard before any update is made. The classification logic is **rule-based**: it uses conditions and decision logic configured in Flow Designer, not machine learning or generative AI. The purposes of the system are to:

1. **Classify consistently:** apply the same documented rules to every eligible record.
2. **Reduce repetitive effort:** fill blank classification fields automatically when the evidence is clear.
3. **Protect existing data:** never overwrite a classification value that is already present.
4. **Avoid guesswork:** route ambiguous, conflicting and unmatched records to review instead of forcing a value.
5. **Stay within scope:** leave assignment, approval, fulfilment and closure to the processes that already handle them.

1.2 Scope of the Project

The scope defines what the initial version of the solution covers, what it intentionally excludes and the assumptions on which the design depends.

Included in the current scope

- Rule-based classification of eligible **Incident** records using the short description and other structured fields confirmed in the instance.
- Rule-based classification of **Requested Items** raised from **existing** Service Catalog items, using the catalog item as the primary signal and catalog variables only where the item alone is not enough.
- Documented rule order, precedence between overlapping rules and explicit handling of conflicts.
- Field-level preservation: writing only blank target fields and leaving populated fields unchanged.
- A review path for ambiguous, conflicting, unmatched or otherwise unsafe cases.
- A planned testing strategy with functional, boundary, regression and repeat-execution test cases.

Excluded from the current scope

- Automatic assignment or reassignment of support groups.
- Automatic **approval, rejection, fulfilment or closure** of service requests, and any change to existing approval or fulfilment processes.
- Creation of new catalog items or redesign of existing ones.
- Machine-learning, predictive or generative-AI classification.
- Claims about production performance, accuracy or time savings, since no measurements have been made.

Assumptions and constraints

- An authorised ServiceNow instance (for example, a personal developer instance or one provided by the institution) will be available for configuration and testing.
- The project account will hold a role that allows flows to be created and tested; ServiceNow documentation lists the flow_designer or admin role for adding actions to a flow [3].
- Incident classification fields, their valid choices and the existing catalog items can be inspected before rules are finalised.
- The review mechanism will be chosen from capabilities that actually exist in the instance; it is not assumed in advance.

1.3 Technologies Used

The project is built entirely on the ServiceNow platform. No external database, separate front end or custom back-end service is required. Table 1.2 lists the components and their role in this project.

Table 1.2: Technologies and platform components

Component	Role in this project
ServiceNow platform	Hosts the Incident and Service Catalog records and provides the configuration environment.
Flow Designer (Workflow Studio)	Low-code builder used to create the two classification flows from triggers, flow logic and actions [1].
Record and Service Catalog triggers	Decide when each flow starts and which record becomes the trigger record [2], [4].
Flow logic (If / Else If, Set Flow Variables)	Evaluate rule conditions, hold match flags and branch between classify and review paths.
Core actions (Look Up Record, Get Catalog Variables, Update Record)	Read the catalog item and its variables, and write the approved classification to blank fields [3], [7].
Incident Management	Source of Incident records and their classification fields [5].
Service Catalog	Source of existing catalog items and the Requested Items raised from them.
Flow execution details	Show which branches ran and which fields changed; used as evidence during testing.

Flows in Flow Designer use **data pills** to pass values such as the trigger record and its fields between steps. Flow variables can store intermediate results, such as whether a particular rule matched. Because the logic is expressed as visible conditions in the flow, an administrator can read each rule and understand why a record was or was not classified.

1.4 Project Objectives

The objectives below are written so that each one can later be checked against the planned test cases in Chapter 4.

1. Design an Incident classification flow that evaluates the short description and other confirmed structured fields against documented rules.
2. Design a Requested Item classification flow that uses the existing catalog item as its primary signal and reads catalog variables only when required.
3. Fill target classification fields only when they are blank, and preserve every non-empty value.
4. Detect unmatched, conflicting and invalid outcomes and send such records to a review path without speculative updates.
5. Keep assignment groups, approvals, fulfilment and closure unchanged by the classification flows.
6. Avoid repeated or recursive updates caused by the flow re-triggering on its own changes.
7. Maintain a rule register that links each rule to its conditions, outputs, exclusions and test cases.
8. Prepare a repeatable testing strategy that can verify each objective once the flows are configured.

1.5 Target Audience

The design is intended for the following groups:

- **Service desk agents**, who currently classify new incidents and would spend less time on routine cases.
- **IT support teams**, who rely on consistent categories for queue views and reports.
- **ServiceNow administrators**, who will configure, test and maintain the flows and the rule register.
- **Service Catalog administrators**, who own the catalog items and variables used in the request mappings.
- **Project evaluators**, who will review the design, scope control and planned testing approach.

1.6 Benefits of the System

Once configured and verified, the design is expected to provide the following benefits. These are design intentions rather than measured results.

- **Consistency:** records with the same clear evidence follow the same rule and receive the same classification.
- **Reduced repetitive work:** clear, routine records do not need manual classification.
- **Data preservation:** existing classification values are never overwritten by the flow.
- **Safe exception handling:** uncertain records are not forced into a category.
- **Maintainability and explainability:** rules are visible conditions in Flow Designer, documented in a rule register, and every automatic outcome can be traced to a rule ID.
- **Process continuity:** approval and fulfilment of requests continue exactly as they do today.

2. SYSTEM REQUIREMENTS

This chapter defines what the classification workflow must do (functional requirements), the qualities it must have (non-functional requirements) and the hardware, software, network and data it depends on. Requirements that depend on the target instance are marked as subject to confirmation.

2.1 Functional Requirements

Each functional requirement in Table 2.1 has an identifier and an acceptance condition. The identifiers are reused in the test plan so that every requirement can be traced to at least one test.

Table 2.1: Functional requirements

ID	Requirement	Acceptance condition
FR-01	The Incident flow shall run only for eligible Incident records under its configured trigger conditions.	An eligible incident starts the flow; an out-of-scope record does not.
FR-02	The Request flow shall run for Requested Items raised from existing catalog items included in the mapping table.	A mapped item is evaluated; an unmapped item goes to review without an update.
FR-03	Incident rules shall evaluate the short description and other structured fields confirmed in the instance.	Clear test inputs follow the expected rule ID.
FR-04	Request rules shall use the catalog item first and read catalog variables only when the mapping requires them.	Variables are read only for items marked as needing them.
FR-05	Rules shall be evaluated independently, and documented precedence shall resolve compatible overlaps.	A specific rule suppresses its broader relative as documented.
FR-06	The flow shall classify a record only when exactly one valid outcome remains.	Zero or several outcomes lead to the review path.
FR-07	The flow shall write only to target fields that are blank.	A blank field is filled for a clear match.
FR-08	The flow shall never overwrite a populated classification field.	Existing values are unchanged after execution.
FR-09	The flow shall not write a value that is absent from the instance choice list or incompatible with existing values.	Invalid or incompatible candidates are not written.
FR-10	Ambiguous, conflicting and unmatched records shall be sent to the review path confirmed in the instance.	The review branch runs and no classification is written.
FR-11	The flows shall not change assignment group, approval, stage, state or closure fields.	Execution details show no change to these fields.
FR-12	The flows shall avoid unnecessary or repeated updates of the same record.	Repeat processing causes no further changes and no loop.

2.2 Non-Functional Requirements

The non-functional requirements describe qualities of the solution. No numeric performance targets are stated, because no measurements have been taken; performance will only be observed during testing.

Table 2.2: Non-functional requirements

Attribute	Requirement
Reliability	The same clear input should always follow the same documented rule, given the same instance configuration.
Data integrity	Non-empty classification values must be preserved, and only valid, compatible values may be written.
Safety	Insufficient or conflicting evidence must never produce a forced classification.
Maintainability	Rules, mappings and review behaviour must be documented in the rule register and readable in the flow.
Traceability	Each automatic outcome should be explainable by a rule ID and visible in the flow execution details.
Scope isolation	The flows must not interfere with existing assignment, approval or fulfilment processes.
Compatibility	The configuration must use only fields, choices, catalog items and variables that exist in the target instance.
Security and access	The flows must operate within the roles and permissions granted to the project account.
Performance	Execution time may be observed during testing; no target or improvement is claimed before measurement.

2.3 Hardware Requirements

Because the ServiceNow platform is cloud-hosted, the project does not require a student-managed server. The local hardware only needs to run a modern web browser comfortably, as summarised in Table 2.3.

Table 2.3: Hardware requirements for the development workstation

Component	Practical requirement
Computer	Laptop or desktop able to run a current, supported web browser.
Processor	Any recent dual-core or better processor.
Memory (RAM)	8 GB is a practical baseline for running the browser and documentation tools together.
Display	A screen of at least 1366 × 768; a larger display makes Flow Designer branches easier to read.
Storage	Enough free space for the report, exported evidence and screenshots taken during testing.
Server infrastructure	Provided by the ServiceNow instance; no local server is assumed.

2.4 Software and Platform Requirements

Table 2.4: Software and platform requirements

Item	Requirement	Status
ServiceNow instance	An authorised instance with Flow Designer (Workflow Studio) available.	To be confirmed
User role	A role permitting flow creation and testing, such as flow_designer or admin [3].	To be confirmed
Incident Management	Incident table with category and subcategory fields and their choice lists.	Inspect in instance
Service Catalog	Existing catalog items in scope, their variables and their current fulfilment flows.	Inspect in instance
Service Catalog support in flows	Ability to use catalog triggers and the Get Catalog Variables action [4], [7].	To be confirmed
Web browser	A current version of a browser supported by the instance release.	Available
Office software	A word processor for the report, rule register and test records.	Available

No custom scripting is planned. If a requirement later proves impossible without a script, the need will be documented and approved before any script is added.

2.5 Network Requirements

- Reliable internet access to reach the authorised ServiceNow instance over HTTPS.
- Valid credentials for the instance and access permitted by the institution's network policy.
- Awareness that developer instances may hibernate or time out when idle; any such interruptions during testing should be noted in the test record.
- Any VPN or proxy requirement imposed by the institution should be documented if encountered.

2.6 Data Model and Record Design

The project does not create a separate database. It operates on records that ServiceNow already stores. The design therefore focuses on which records and fields the flows read, which they may write, and which they must never change. Table 2.6 lists the records involved; table names shown in brackets are standard ServiceNow tables, but the exact fields used must still be confirmed in the target instance.

Table 2.5: Records and data sources used by the flows

Record / data source	Role in the workflow	Handling
Incident [incident]	Provides the short description, current category and subcategory, and other structured fields.	Read for rule evaluation; only blank classification fields may be written.
Requested Item [sc_req_item]	Represents one ordered catalog item and becomes the trigger record for the request flow.	Read; the target classification field must be confirmed before any write.
Catalog Item [sc_cat_item]	Identifies which existing catalog item was ordered.	Read only; used as the primary mapping key.
Catalog variables	Provide answers submitted with the request, where the item alone is not enough.	Read only, through the Get Catalog Variables action [7].
Choice lists	Define the valid category and subcategory values.	Read during design; every rule output must exist here.
Flow variables	Hold match flags, the candidate outcome and the reason for review.	Internal to the flow; not stored on the record.

An important design point concerns the target field on Requested Items. Incident records include category and subcategory fields as part of Incident Management, but a Requested Item does not necessarily have an equivalent classification field in every instance. Before the request flow is configured, the administrator must confirm which existing field should hold the classification. If no suitable field exists, adding one is a configuration change that requires the instance owner's approval; this report does not assume that such a field is available.

Table 2.7 summarises the read and write permissions that the flows are designed to respect.

Table 2.6: Field-level read and write matrix

Field	Incident flow	Request flow
Short description	Read	Read (supporting evidence)
Category	Read; write only if blank	Not applicable
Subcategory	Read; write only if blank and compatible	Not applicable
Catalog item / variables	Not applicable	Read
Request classification field (to be confirmed)	Not applicable	Read; write only if blank
Work notes (only if chosen as review mechanism)	Append a review note	Append a review note
Assignment group, approval, stage, state, closure	Never written	Never written

The data integrity rule that follows from this matrix is simple: each target field is checked separately, a blank field may receive a valid and compatible value, a populated field is preserved, and a candidate that conflicts with existing data is not written.

3. CODING: FLOW DESIGN AND CONFIGURATION

The HOD's template places source code in this chapter because its sample project is a web application. This project is a low-code ServiceNow workflow, so the equivalent of "coding" is the design and configuration of Flow Designer flows. This chapter therefore presents the architecture, the workflows, the rules, the decision logic and the configuration procedure, together with pseudocode that expresses the intended logic precisely. The pseudocode is a design aid; it is not a ServiceNow script and will not be pasted into the instance.

3.1 Proposed System Architecture

The architecture is organised into four layers, shown in Figure 3.1. **Record sources** are the people who create records: callers who report incidents, service desk agents who log them, and requesters who order existing catalog items. **Platform records** are the Incident and Requested Item records that ServiceNow stores. The **Flow Designer layer** holds the two triggers, the ordered rule conditions, the field-preservation guard and the final action. **Outcomes** are the only four things the design allows to happen.

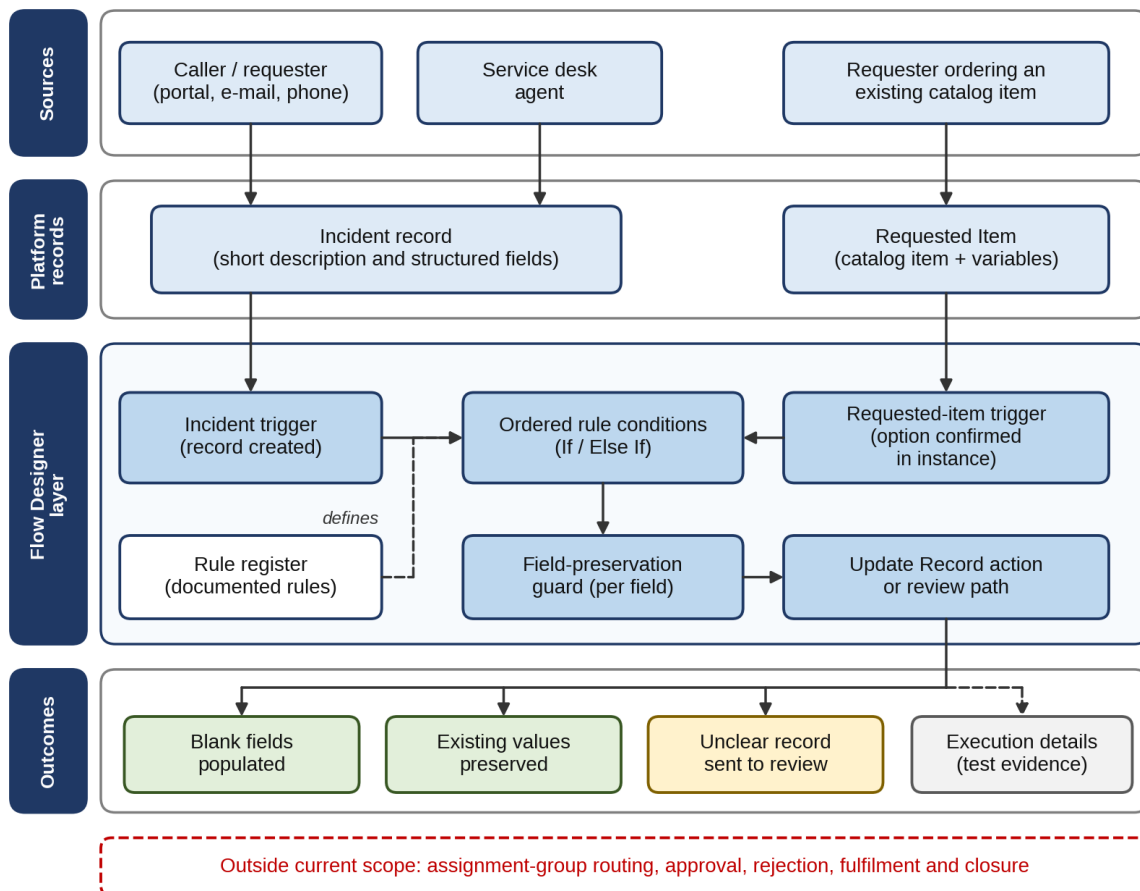


Figure 3.1: Proposed system architecture for rule-based classification

Two features of the architecture deserve emphasis. First, the rule register sits beside the flow as documentation: every condition configured in the flow corresponds to a documented rule ID, so the flow and its explanation cannot drift apart unnoticed. Second, the boundary of the design is drawn explicitly. Assignment-group routing, approval, rejection, fulfilment and closure are shown outside the system, which means no action in either flow is permitted to touch the fields that control them.

3.2 Overall Classification Workflow

Figure 3.2 shows the end-to-end workflow shared by both record types. After an eligible record is created, the record type determines which inputs are read. Both paths then pass through the same sequence of steps: rule evaluation, a single decision on whether exactly one clear and valid classification exists, the field-preservation guard, and finally an outcome that can be verified in the flow execution details.

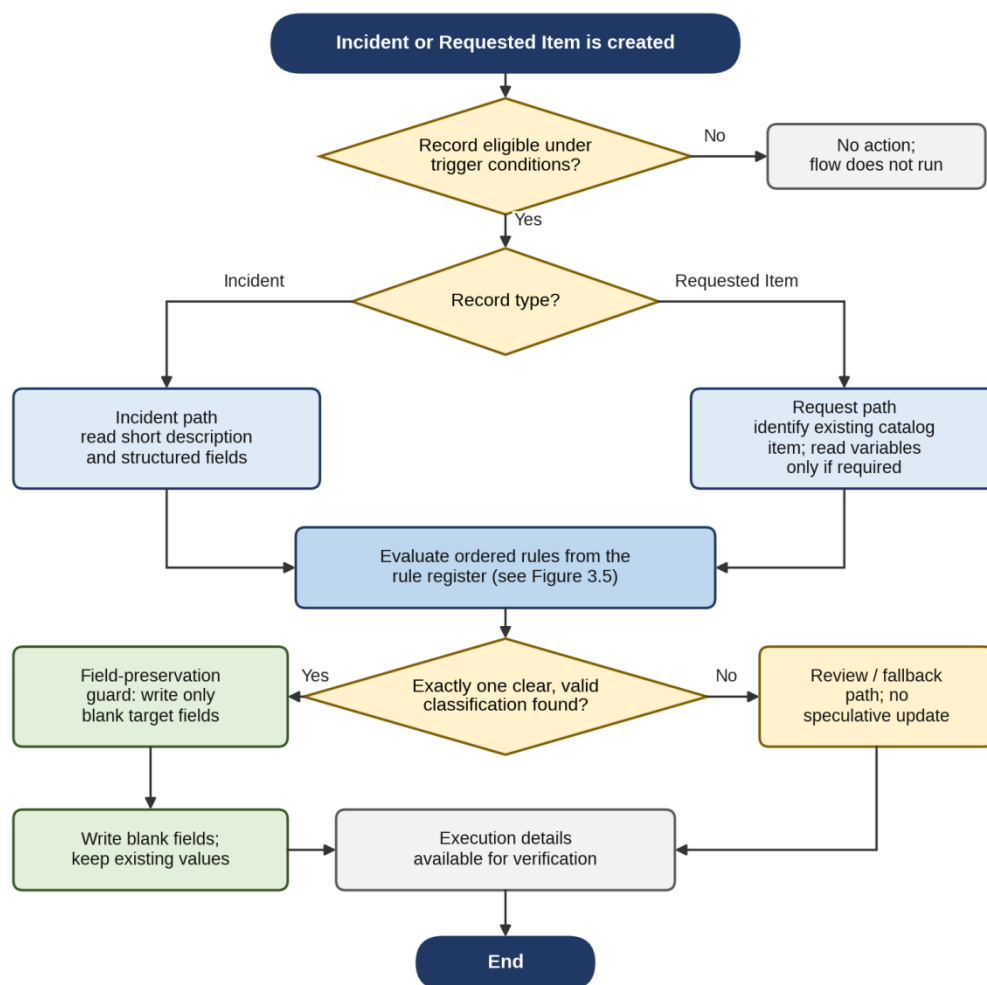


Figure 3.2: Overall classification workflow for Incidents and Requested Items

Although the diagram shows a single workflow for clarity, the design uses **two separate flows**. The two record types have different triggers, different inputs and different risks: an incident is described in free text, whereas a request is defined mainly by the catalog item that was ordered. Separate flows keep each one simpler to read, allow each to be activated and tested independently, and ensure that a change to the request mappings cannot affect incident classification by accident.

3.3 Flow Designer Triggers, Conditions and Actions

Each flow is built from three kinds of component. A **trigger** specifies when the flow runs; when its conditions are met, the system runs the flow using the data provided by the trigger [1]. **Flow logic**, such as If and Else If blocks, decides which path to follow. **Actions** perform work such as looking up or updating records. Table 3.1 lists the components the design relies on.

Table 3.1: Flow Designer components used in the design

Component	Type	Planned use
Record trigger (Created) on Incident	Trigger	Starts the Incident flow when a new incident is created, with a condition that category or subcategory is still blank.
Trigger for Requested Items	Trigger	Starts the request flow; the exact option is chosen as described in Table 3.2.
If / Else If	Flow logic	Checks each rule condition and the final "exactly one outcome" decision.
Set Flow Variables	Flow logic	Stores a match flag per rule, the candidate values and the reason for review.
Look Up Record	Action	Retrieves the catalog item behind a Requested Item where it is not directly available.
Get Catalog Variables	Action	Returns the values of selected catalog variables from the requested item [7].
Update Record	Action	Writes only the approved blank fields, or a review note, to the trigger record [3].

Choosing the trigger for existing catalog items

The request flow needs particular care. ServiceNow documentation explains that a Service Catalog trigger runs for requests of a specific item only when that catalog item is configured to run the flow, and that the requested item then becomes the trigger record. The same page notes that catalog variables cannot be used in the trigger condition and must instead be read in the body of the flow [4]. Since this project classifies **existing** catalog items, many of which may already have their own fulfilment flow attached, simply switching an item to a new classification flow could replace the process that currently fulfils it. The available options are compared in Table 3.2.

Table 3.2: Trigger options for classifying Requested Items from existing catalog items

Option	How it works	Advantages	Concerns
A. Record trigger on the Requested Item table	Flow starts when a Requested Item record is created, filtered to mapped items.	Does not change the catalog item or its existing fulfilment flow.	Availability of this trigger on the table must be confirmed in the instance.
B. Reusable subflow	Classification logic packaged as a subflow and called from existing catalog flows.	Reusable; logic maintained in one place.	Requires editing each existing fulfilment flow, which needs owner approval.
C. Service Catalog trigger	Flow attached to a catalog item through its Flow field [4].	Standard catalog pattern with direct access to the requested item.	Suitable only for items that have no existing flow; otherwise it would replace fulfilment.

Option A is the preferred design because it leaves existing catalog items and their fulfilment untouched. The final choice is **subject to confirmation** once the trigger options available in the instance have been inspected. Whichever option is used, the flow will contain no approval, catalog task or closure actions.

Run behaviour and recursion

ServiceNow documentation warns that when a record-triggered flow changes its own trigger record, that change can satisfy the trigger conditions again and cause recursion [2]. The design avoids this in three ways: the incident trigger uses the **Created** event rather than every update; the trigger condition requires a classification field to be blank; and the field guard performs at most one Update Record action, which fills the blank fields so that the condition is no longer true. Repeat-execution tests in Chapter 4 are planned specifically to confirm this behaviour.

3.4 Rule-Based Incident Classification

The Incident flow, shown in Figure 3.3, first confirms that there is something to classify. If both category and subcategory already hold values, the flow ends without making any change. Otherwise it reads the short description and other confirmed fields and runs five **independent** rule checks. Each check sets its own match flag rather than stopping at the first match. This is a deliberate choice: a simple first-match Else If ladder would silently hide a conflict whenever two unrelated rules both match.

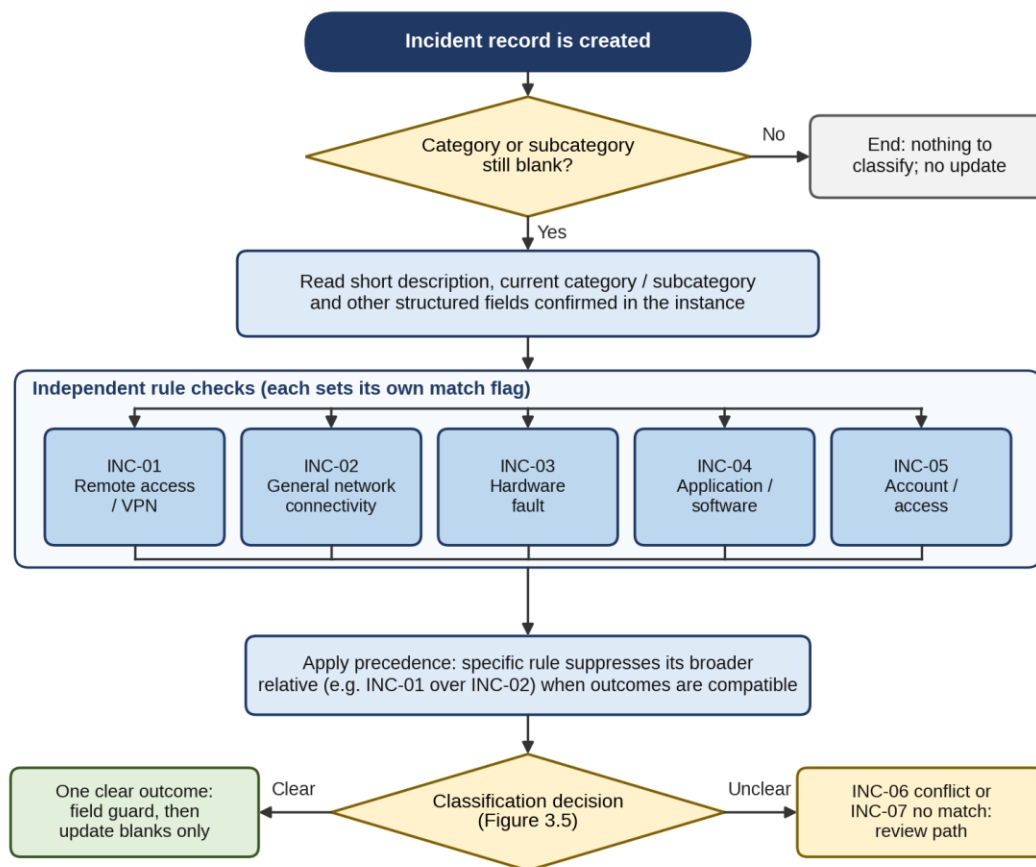


Figure 3.3: Incident classification workflow

Table 3.3 is the initial rule register for incidents. The phrases listed are **illustrative examples** of evidence that might appear in a short description. Outputs written in square brackets, such as [Network], are placeholders: they must be replaced with values confirmed in the instance choice list before configuration, and no value will be written unless it exists there.

Table 3.3: Incident rule register (initial design, illustrative)

Rule	Illustrative evidence	Candidate output	Exclusion / guard
INC-01	"VPN will not connect", "remote access drops"	[Network] / [VPN] if such a subcategory exists	Do not assume a VPN subcategory exists; suppresses INC-02 when both match.
INC-02	"Wi-Fi disconnects", "no network", "cannot reach shared drive"	[Network]	A generic phrase such as "not working" is not enough on its own.
INC-03	"laptop will not power on", "monitor flickers", "keyboard broken"	[Hardware]	Separate physical faults from software configuration problems.
INC-04	"application crashes", "error after update", "install fails"	[Software]	If network evidence is also strong, treat as a conflict.
INC-05	"password reset", "account locked", "cannot sign in"	[Access] if available in the choice list	If no suitable category exists, send to review instead.
INC-06	Two or more incompatible rules remain after precedence	None	Conflict: preserve values and use the review path.
INC-07	No rule matches, or the description is too vague	None	Unmatched: preserve values and use the review path.

Each rule is configured as a group of conditions on the short description, for example "short description contains VPN" OR "short description contains remote access". Combinations can also require two signals together where one word alone would be too broad. Precedence is applied only between **compatible** rules: a VPN problem is a more specific kind of network problem, so INC-01 suppresses INC-02 and the record is not treated as a conflict. Rules with incompatible outputs, such as network and software, are never resolved by order alone. Algorithm 3.1 states this logic precisely.

Algorithm 3.1: Incident rule evaluation

```

INPUT  incident record r
IF r.category is not blank AND r.subcategory is not blank THEN
  STOP                                     // nothing to classify
text <- r.short_description
FOR EACH rule k IN {INC-01 .. INC-05}:  // independent checks
  match[k] <- conditions_of(k) are satisfied by text and fields
IF match[INC-01] AND match[INC-02] THEN
  match[INC-02] <- FALSE                 // specific suppresses broad
outcomes <- distinct outputs of all rules with match = TRUE
IF |outcomes| = 0 THEN send_to_review(r, "INC-07 unmatched")
ELSE IF |outcomes| > 1 THEN send_to_review(r, "INC-06 conflict")
ELSE classify(r, the single outcome)     // see Algorithm 3.3

```

3.5 Classification of Existing Service Catalog Request Items

Requests are handled differently from incidents. A request is defined mainly by the catalog item that was ordered, so the existing catalog item is the most reliable signal and the primary mapping key. Free-text matching is not used as the main mechanism for requests. Figure 3.4 shows the request workflow.

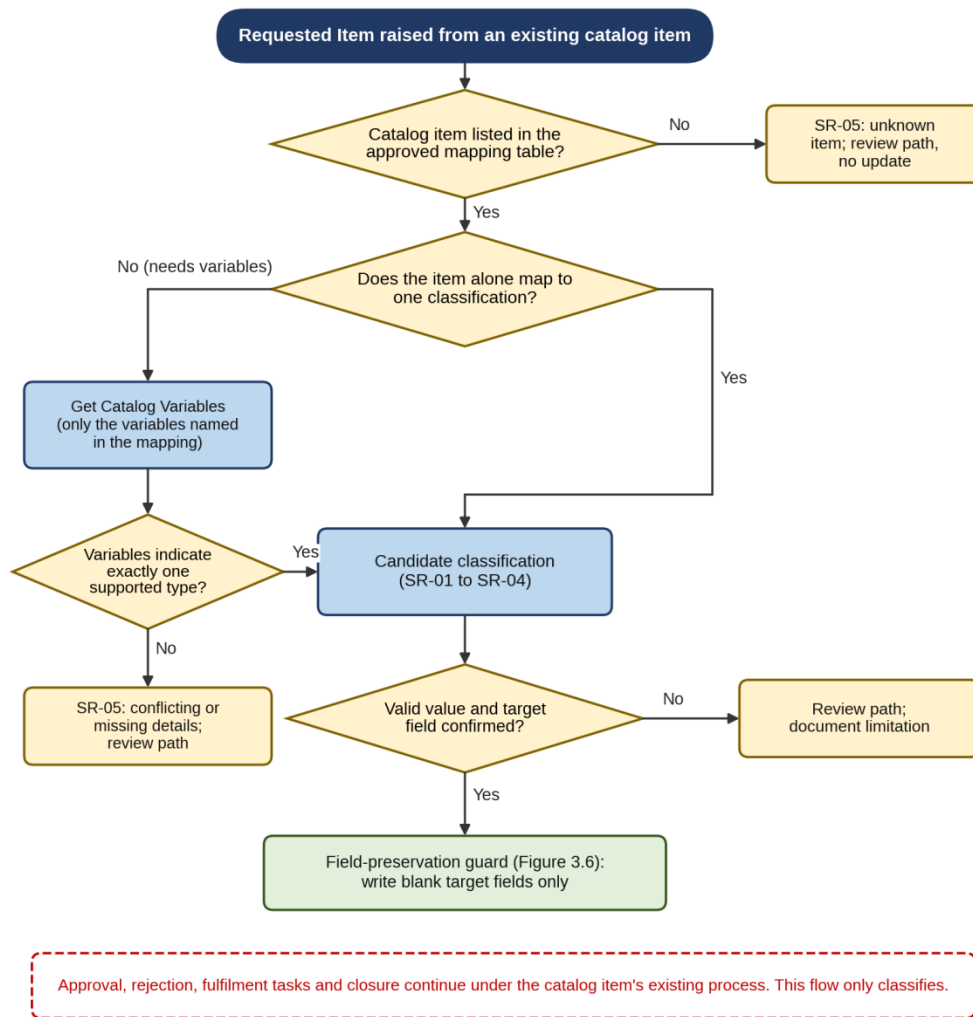


Figure 3.4: Classification workflow for Requested Items from existing catalog items

The flow first checks whether the ordered item appears in an approved **mapping table** maintained in the rule register. An unmapped item is never guessed; it goes to review. For a mapped item, the mapping states whether the item alone determines the classification. Where one catalog item covers several kinds of request, the mapping names the specific catalog variables that distinguish them, and only those variables are read with the Get Catalog Variables action. ServiceNow documentation notes that this action returns variable values as they were at the time the action ran [7], which is suitable here because classification happens once, when the request is created.

Table 3.4: Request mapping rules (initial design, illustrative)

Rule	Mapping condition	Candidate output	Notes
SR-01	Ordered item is a mapped hardware or equipment item	[Hardware request]	Item names to be listed after catalog inspection.
SR-02	Ordered item is a mapped software installation or licence item	[Software request]	Item names to be listed after catalog inspection.
SR-03	Ordered item is a mapped account, access or permission item	[Access request]	Item names to be listed after catalog inspection.
SR-04	Item needs variables, and the named variables indicate exactly one supported type	As defined in the mapping	Only variables listed in the mapping are read.
SR-05	Item unmapped, variables missing or conflicting, or target field unconfirmed	None	No update; review path.

No real catalog item names appear in Table 3.4 because they have not yet been inspected in the instance. They will be added to the mapping table only after the catalog owner has confirmed each mapping. Algorithm 3.2 expresses the request logic.

Algorithm 3.2: Requested Item classification

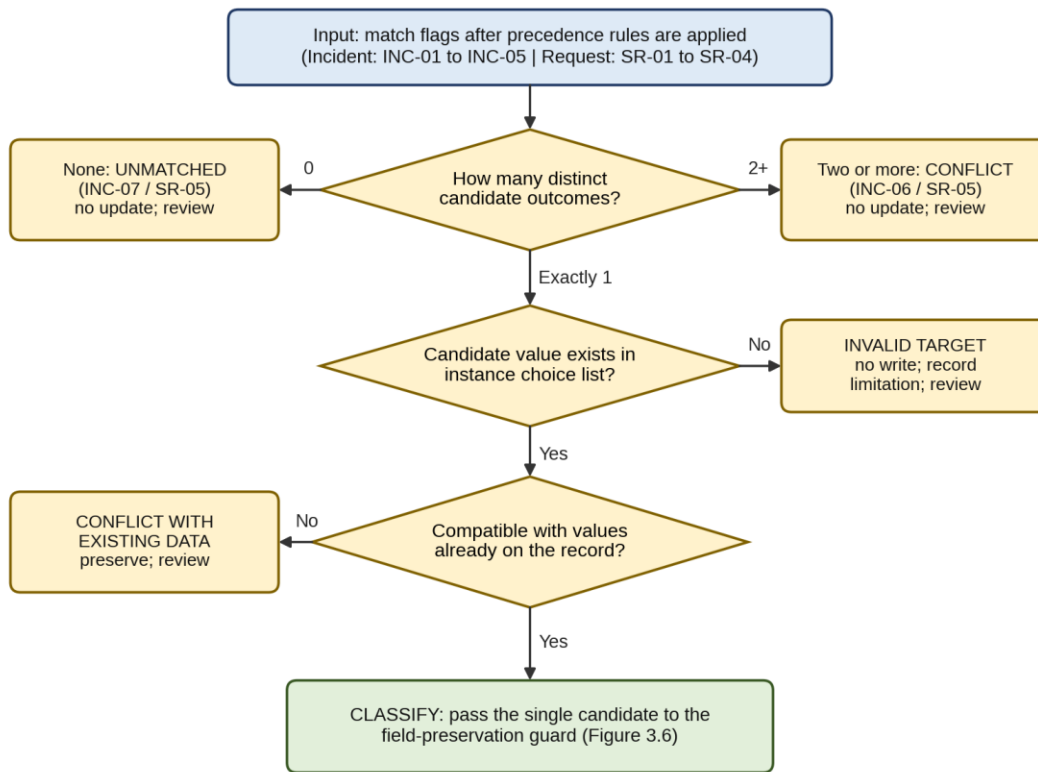
```

INPUT  requested item q
item <- catalog item of q
IF item NOT IN mapping_table THEN send_to_review(q, "SR-05 unknown item"); STOP
m <- mapping_table[item]
IF m.uses_variables = FALSE THEN
    candidate <- m.output                // SR-01 .. SR-03
ELSE
    v <- Get Catalog Variables(q, m.variable_list)
    options <- outputs in m whose variable conditions are met by v
    IF |options| <> 1 THEN send_to_review(q, "SR-05 unclear details"); STOP
    candidate <- the single option        // SR-04
IF target field or candidate value is not confirmed THEN
    send_to_review(q, "target not confirmed"); STOP
classify(q, candidate)                  // see Algorithm 3.3
// approval, catalog tasks, fulfilment and closure are never touched

```

3.6 Classification Decision Process

Both flows end their rule evaluation with the same decision process, shown in Figure 3.5. The process is designed so that every possible situation leads to one of five named outcomes, and only one of those outcomes, **CLASSIFY**, allows a value to be written.



Review means the record is left for a person to classify through the review mechanism confirmed in the instance (for example, an existing triage queue or a work note).

Figure 3.5: Classification decision process

The checks are ordered from the cheapest to the most specific. Counting distinct outcomes catches the two most common unclear cases first. The choice-list check ensures that a rule cannot write a value the instance does not recognise, which protects against rules drifting out of date when categories are renamed. The compatibility check protects partially classified records: for example, if an agent has already set a category, the flow must not add a subcategory that belongs to a different category.

Handling ambiguous, conflicting and unmatched records

Table 3.5 lists the unclear situations the design recognises and the safe behaviour for each. In every case the classification fields are left exactly as they were.

Table 3.5: Handling of ambiguous, conflicting and unmatched records

Situation	Example	Safe behaviour
Unmatched	"It does not work" with no other detail.	No update; review path with reason INC-07 / SR-05.
Conflict between rules	"Cannot open the app because VPN is down" matches network and software.	No update; review path with reason INC-06.
Conflict with existing data	Category already set to a value incompatible with the candidate subcategory.	Preserve existing values; review path.
Invalid target value	Candidate value has been removed from the choice list.	Do not write; record the limitation; review path.
Missing input	A required catalog variable is empty or cannot be read.	Do not classify from missing data; review path.
Permission failure	The flow cannot read or update a required field.	Do not claim success; document the error and role context.

The review path itself must be confirmed in the instance rather than invented. The simplest acceptable option is to leave the classification fields blank, so that the record remains in whatever triage view the service desk already uses, and to append a short work note naming the reason, such as "Auto-classification skipped: INC-06 conflict". Because the work note is informational, it does not change assignment or state.

3.7 Record-Field Handling and Preservation of Existing Values

The field-preservation guard, shown in Figure 3.6, is the safety mechanism that makes the design acceptable for live records. It examines each target field separately, in a fixed order, and builds a list of pending changes. Only after both fields have been checked does it decide whether any update is needed.

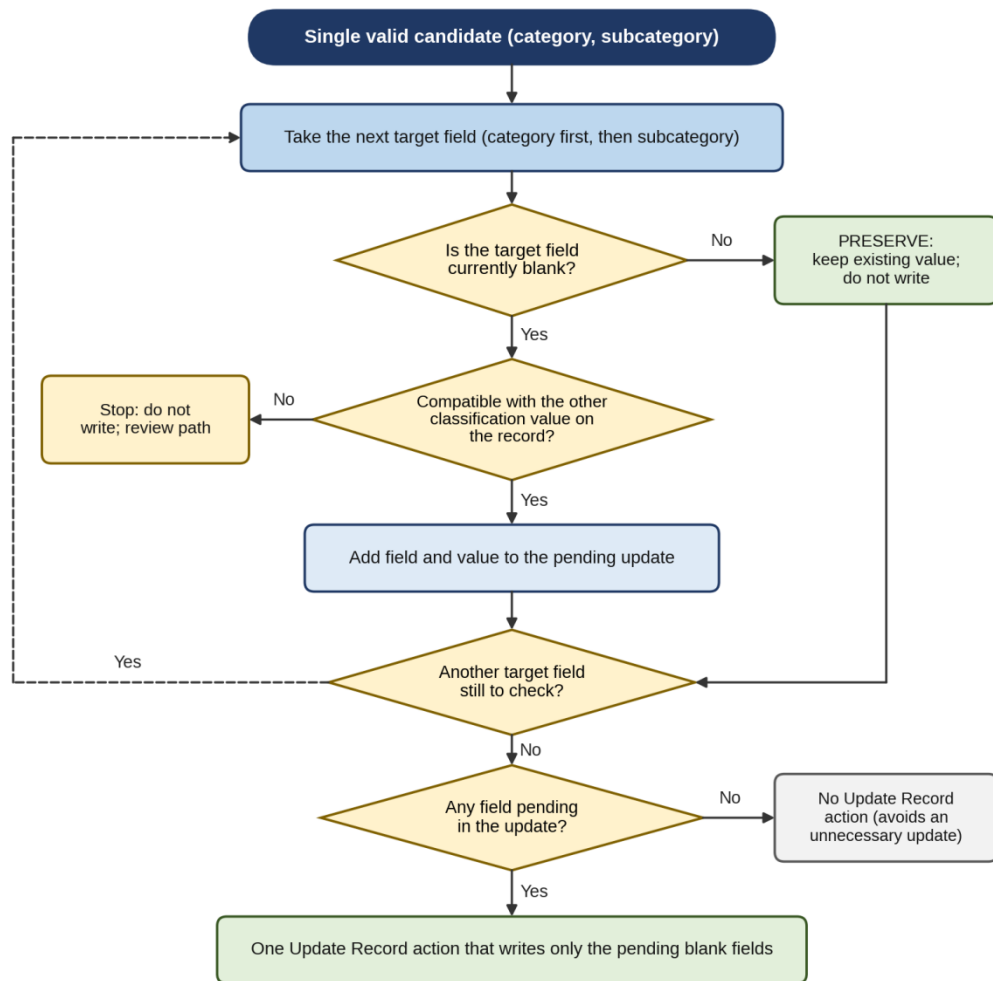


Figure 3.6: Field-preservation logic applied to each target field

For incidents, the four possible starting states of the category and subcategory fields lead to the behaviour shown in Table 3.6. The same principle applies to the single classification field on Requested Items once that field has been confirmed.

Table 3.6: Field-preservation behaviour for each starting state

Category	Subcategory	Behaviour of the flow
Blank	Blank	Write both candidate values in a single Update Record action.
Populated	Blank	Keep the category; write the subcategory only if it belongs to the existing category.
Blank	Populated	Keep the subcategory; write the category only if it is the parent of the existing subcategory; otherwise review.
Populated	Populated	No change; the trigger condition normally prevents the flow from running at all.

Algorithm 3.3: Field-preservation guard

```
PROCEDURE classify(record x, candidate c)
  pending <- empty map
  FOR EACH field f IN [category, subcategory]: // fixed order
    IF x.f is not blank THEN CONTINUE // preserve
    IF c.f not compatible with the other value on x THEN
      send_to_review(x, "conflict with existing data"); RETURN
    pending[f] <- c.f
  IF pending is empty THEN RETURN // no needless update
  Update Record(x, pending) // one write only
  // assignment group, state, approval and closure are never in pending
```

Writing all pending fields in one Update Record action, and skipping the action entirely when nothing is pending, keeps the number of updates to a minimum. This reduces the chance of re-triggering other automation and keeps the record history easy to read.

3.8 Implementation Methodology and Configuration Steps

The implementation will follow an incremental, test-driven approach: each rule will be configured together with its test case, verified, and only then followed by the next rule. Table 3.7 shows the planned phases. None of these phases has yet been carried out.

Table 3.7: Planned implementation phases

Phase	Activity	Exit condition
1. Discovery	Confirm instance access, roles, incident fields and choice lists, catalog items, variables and existing fulfilment flows.	Confirmed field and catalog inventory recorded.
2. Rule design	Finalise the rule register and mapping table with real values; resolve overlaps; link each rule to tests.	Rule register reviewed by the guide.
3. Incident flow	Configure trigger, rule checks, decision process, field guard and review path.	Incident test cases ready to run.
4. Request flow	Select the trigger option, configure item mapping, variable reads and field guard.	Request test cases ready to run.
5. Testing	Execute the planned tests and record genuine evidence.	All results recorded, defects logged.
6. Review	Refine rules, update documentation and add evidence to the report.	Report updated with actual results.

Configuration steps for the Incident flow

1. Open Flow Designer and create a new flow with a descriptive name that follows the instance's naming convention, for example "Auto Classify Incident".
2. Add a record trigger on the Incident table for the Created event, with the condition that category is empty or subcategory is empty.
3. Add Set Flow Variables steps to initialise one Boolean match flag per rule, plus variables for the candidate values and review reason.
4. Add one If block per rule INC-01 to INC-05, each setting its own flag when its conditions on the short description are met.
5. Add the precedence step that clears the INC-02 flag when INC-01 has matched.

6. Add the decision block: zero or several outcomes lead to the review path; exactly one leads to the choice-list and compatibility checks.
7. Add the field-guard logic and a single Update Record action that sets only the pending blank fields.
8. Configure the review path using the mechanism confirmed in the instance, such as a work note.
9. Save the flow, test it with prepared records, and inspect the execution details before activating it in the test instance.

Configuration steps for the Request flow

1. Inspect each existing catalog item in scope, record whether it already has a fulfilment flow, and list the variables relevant to classification.
2. Confirm the target classification field on Requested Items with the instance administrator.
3. Create the flow using the trigger option selected from Table 3.2, restricted to mapped catalog items.
4. Add a Look Up Record step, if needed, to obtain the catalog item, and an If block that checks it against the mapping table.
5. For items that need variables, add Get Catalog Variables with only the variables named in the mapping [7].
6. Reuse the same decision process, field guard and review path as the Incident flow.
7. Verify that the flow contains no approval, catalog task or closure actions, then test and inspect the execution details.

3.9 Expected Outputs

The template includes an output section with screenshots. Because the flows have not been executed, no screenshots are included, and none have been simulated. Instead, Table 3.8 describes the **expected** state of representative records before and after the flow runs. These are design expectations that the tests in Chapter 4 will check; genuine screenshots will be added to Appendix A only after execution.

Table 3.8: Expected record states for representative scenarios (illustrative)

Scenario	Before	Expected after
Incident: "VPN disconnects every hour"	Category and subcategory blank	Category [Network]; subcategory [VPN] if it exists; rule INC-01
Incident: "Laptop will not power on"	Category blank; subcategory blank	Category [Hardware]; rule INC-03
Incident: same text, category already set by agent	Category populated	Category unchanged; compatible blank subcategory only
Incident: "App fails because VPN is down"	Both blank	Both still blank; review note INC-06
Request: mapped software item	Classification field blank	Field set to [Software request]; rule SR-02; approval unchanged
Request: unmapped item	Classification field blank	Field still blank; review note SR-05

4. TESTING

Testing will verify that each rule produces its expected outcome, that existing values are preserved, that unclear records are handled safely and that the flows do not interfere with other processes. **All test cases in this chapter are planned. None has been executed**, and no results, pass rates or measurements are reported. The status of every case is therefore shown as "Planned" until it is run in the authorised instance.

4.1 Testing Strategy

Several complementary kinds of testing are planned, as shown in Table 4.1. Together they cover both what the flows should do and what they must never do.

Table 4.1: Planned testing types

Testing type	Purpose	Scope in this project
Rule-level (unit) testing	Confirm that each rule condition matches the evidence it is meant to match.	One positive and at least one negative example per rule.
End-to-end flow testing	Confirm that a real record passes through trigger, decision, guard and update correctly.	Prepared Incident and Requested Item records.
Boundary testing	Exercise edge states of the classification fields and inputs.	Partially filled fields, empty variables, vague text.
Negative testing	Confirm that unsafe cases never produce a write.	Conflicts, unknown items, invalid choice values.
Regression testing	Confirm that existing processes behave as before.	Approvals, fulfilment and assignment on requests.
Repeat-execution testing	Confirm there is no recursion or repeated updating.	Updating a record after it has been classified.

4.2 Test Environment and Preconditions

- Tests will be run only in the authorised non-production instance, never on live service desk records.
- Test records will use fictitious callers and descriptions, so that no personal or sensitive data appears in evidence.
- The rule register and mapping table will be finalised with confirmed values before testing begins.
- For each test, the tester will record the record number, input values, expected result, observed result and the flow execution reference.
- Flow execution details, which show the steps that ran and the values involved, will be used as the primary evidence rather than assuming that a saved record proves correct behaviour.

4.3 Planned Test Cases

Table 4.2 lists the planned test cases. The last column traces each case to the functional requirements in Table 2.1.

Table 4.2: Planned test cases (not yet executed)

ID	Input / condition	Expected result	FR	Status
TC-01	Incident with clear VPN wording; both fields blank	INC-01 output written to blank fields only	FR-03, 07	Planned
TC-02	Incident with clear Wi-Fi / network wording	INC-02 output written	FR-03, 07	Planned
TC-03	Incident with VPN and general network wording	INC-01 applied; not treated as conflict	FR-05	Planned
TC-04	Incident with clear hardware fault	INC-03 output written	FR-03	Planned
TC-05	Incident with clear application crash	INC-04 output written	FR-03	Planned
TC-06	Incident with account lockout wording	INC-05 output written, or review if no access category exists	FR-03, 09	Planned
TC-07	Both category and subcategory already populated	Flow does not run or makes no change	FR-01, 08	Planned
TC-08	Category populated, subcategory blank, compatible rule	Only the compatible subcategory is written	FR-07, 08	Planned
TC-09	Category blank, subcategory populated, incompatible candidate	No write; review path	FR-09, 10	Planned
TC-10	Vague incident: "it does not work"	No write; review reason INC-07	FR-06, 10	Planned
TC-11	Network and software wording together	No write; review reason INC-06	FR-06, 10	Planned
TC-12	Requested Item from a mapped item that needs no variables	Mapped output written to blank field	FR-02, 07	Planned
TC-13	Mapped item whose variables indicate one type	SR-04 output written	FR-04	Planned
TC-14	Mapped item with conflicting or empty variables	No write; review path	FR-06, 10	Planned
TC-15	Requested Item from an unmapped catalog item	No write; review reason SR-05	FR-02, 10	Planned
TC-16	Request that requires approval	Approval and fulfilment behave exactly as before	FR-11	Planned
TC-17	Classified record updated again for an unrelated reason	No further classification update; no loop	FR-12	Planned
TC-18	Candidate value missing from the choice list	Value not written; limitation recorded	FR-09	Planned

4.4 Detailed Test Case Descriptions

Following the format of the HOD's template, three representative cases are described in more detail below. The remaining cases will be documented in the same way in the test record (Appendix A).

Table 4.3: Detailed description of TC-01

Element	Description
Scenario	A user reports a remote-access problem and no classification has been entered.
Preconditions	Incident flow saved in the test instance; INC-01 configured; [Network] and, if present, [VPN] confirmed in the choice lists.
Steps	1. Create an incident with short description "VPN disconnects every few minutes". 2. Leave category and subcategory blank. 3. Save. 4. Open the flow execution details.
Expected outcome	INC-01 flag is true and INC-02 is suppressed; the confirmed values are written to the blank fields; assignment group is unchanged.

Table 4.4: Detailed description of TC-08

Element	Description
Scenario	An agent has already chosen a category, but no subcategory, before the flow evaluates the record.
Preconditions	Category set to the confirmed [Network] value; subcategory blank.
Steps	1. Create the incident with a clear VPN description and the category already set. 2. Save. 3. Compare the record before and after; inspect execution details.
Expected outcome	The category is unchanged; only a subcategory that belongs to the existing category is written.

Table 4.5: Detailed description of TC-16

Element	Description
Scenario	A mapped catalog item that normally requires approval is ordered.
Preconditions	Request flow active for mapped items; the item's existing fulfilment process unchanged.
Steps	1. Order the item through the catalog. 2. Observe the approval and fulfilment steps. 3. Inspect the request flow execution details.
Expected outcome	Only the classification field is written; approval, catalog tasks, stage and closure follow the existing process exactly as before.

4.5 Evaluation Approach and Acceptance Criteria

After execution, results will be reported as simple counts: the number of clear-match cases that produced the expected output, the number of unclear cases correctly left unchanged, the number of preservation tests passed and the number of unintended updates observed. These describe the test set only and will not be presented as accuracy across real production records. No percentage or timing figure will be stated unless the sample, the expected and observed values and the calculation method are recorded alongside it. The design will be considered ready for demonstration when:

- every rule has at least one passing positive test and one passing negative or boundary test;
- every value written by the flows exists in the instance choice lists;
- no populated classification value is ever overwritten;
- no unclear record receives a speculative value;
- assignment groups, approvals and fulfilment are unaffected;
- repeat processing causes no further updates; and
- all evidence consists of genuine, correctly labelled screenshots and execution details.

5. TECHNICAL CHALLENGES

This chapter identifies the technical challenges expected during implementation and the approach planned for each. They are anticipated challenges; the chapter will be revised to describe actual issues and resolutions after configuration and testing.

5.1 Variation in Free-Text Descriptions

Challenge: Users describe the same problem in many ways, and a single keyword can appear in unrelated contexts. "Network" might appear in a description of a software licence problem, for instance.

- Use groups of phrases rather than single words, and require two signals where one alone would be too broad.
- Record a positive and a negative example for each rule, and refine wording only on the basis of test evidence.
- Accept that some records will remain unmatched; sending them to review is the intended behaviour, not a failure.

5.2 Overlapping and Conflicting Rules

Challenge: Rules can overlap. If the flow simply picks the first match, real conflicts become invisible and the result depends on rule order.

- Evaluate all rules independently with separate match flags, then count distinct outcomes.
- Allow precedence only between compatible rules, document it in the rule register and test it explicitly (TC-03).
- Route incompatible combinations to review with a named reason (TC-11).

5.3 Variability of Catalog Items and Variables

Challenge: Each catalog item has its own variables, and the same idea may be captured differently in different items. Variables cannot be used in a Service Catalog trigger condition [4].

- Use the catalog item as the primary key and read variables only for items that the mapping marks as needing them.
- Read variables in the body of the flow with Get Catalog Variables, naming only the required variables [7].
- Review the mapping table whenever catalog items are added, renamed or retired.

5.4 Trigger Selection for Existing Catalog Items

Challenge: Attaching a new flow to an existing catalog item could replace the flow that currently fulfils it, which would break the requirement to leave fulfilment unchanged.

- Prefer a record trigger on Requested Items, which leaves catalog items untouched (Option A in Table 3.2).
- Use a subflow or a Service Catalog trigger only where the item owner agrees and the consequences are understood.

- Run regression test TC-16 on every item before and after the flow is activated.

5.5 Flow Re-triggering and Repeated Updates

Challenge: A flow that updates its own trigger record can satisfy its trigger conditions again and cause recursion [2]. Other automation on the same table may also react to each update.

- Trigger on record creation, with a condition that a classification field is blank.
- Combine all changes into a single Update Record action, and skip the action when nothing is pending.
- Confirm the behaviour with repeat-execution test TC-17 and by reviewing execution history.

5.6 Instance Permissions and Unverified Configuration

Challenge: Field names, choice values, catalog items, trigger options and roles all depend on the instance, and none has been confirmed yet.

- Complete the discovery phase before any configuration, and record the confirmed inventory.
- Keep every placeholder visible in square brackets until it is replaced with a confirmed value.
- Document any unavailable feature or permission as a limitation instead of working around it silently.

6. DESIGN CHALLENGES AND CONSIDERATIONS

6.1 Balancing Automation and Control

The central design trade-off is between how many records are classified automatically and how safe each automatic decision is. A design that tries to classify every record will inevitably guess on some of them. This design chooses control: it automates only decisions supported by clear evidence and leaves the rest to people. The cost is that some records will still need manual classification; the benefit is that an automatic value can be trusted.

6.2 Separating Classification from Routing and Fulfilment

Classification describes the nature of an issue or request; assignment decides who handles it; approval and fulfilment decide whether and how a request is delivered. Combining these decisions in one flow would multiply the consequences of a single wrong rule. Keeping classification separate means that a misclassified record still reaches the right people through existing processes, and that routing can be added later as an independent, separately tested step.

6.3 Explainability and Maintainability

A rule-based design is valuable only if people can understand it. Each rule therefore has an identifier, a plain-language description, documented exclusions and linked tests. Review notes name the reason a record was not classified. The rule register and the configured flow must be changed together; a change is complete only when the affected tests have been re-run, not merely when the flow saves successfully.

6.4 Design Considerations and Limitations

The current design has the following known limitations, which are deliberate consequences of its scope:

- Rules rely on explicit conditions, so new wording, uncommon problems or poorly described records may remain unmatched.
- Classification quality for requests depends on how well existing catalog items and variables describe the request.
- The target field for Requested Items, the review mechanism and the trigger option must be confirmed in the instance.
- The design does not route records to support groups, so it does not by itself shorten resolution time.
- No machine-learning capability, accuracy level or production throughput is claimed.

7. FUTURE ENHANCEMENTS

The following enhancements could be considered once the initial classification flows have been configured and verified. Each would need its own requirements, approval and testing.

7.1 Support-Group Routing

Approved category and subcategory combinations could be mapped to support groups, so that a verified classification also suggests or sets the assignment group. This would be added as a separate flow or step, with its own conflict handling and its own regression tests, and only after classification has been shown to behave reliably.

7.2 Easier Rule Management

As the number of rules grows, maintaining many If blocks becomes harder. Rule conditions and outputs could be moved into a table or decision-table feature, if one is available and approved in the instance, so that administrators can update mappings without editing the flow structure. A versioned change log for the rule register would support auditing.

7.3 Expanded Catalog Coverage

More existing catalog items and their variables could be added to the mapping table as business owners confirm the intended classifications. Other record types could be considered once the two initial paths are stable.

7.4 Reporting and Monitoring

Reports could show how many records were classified automatically, how many were sent to review and for what reason, and which rules are used most often. A rising number of review cases for a particular reason would indicate where a new rule or a clearer catalog item is needed. Such reports must respect access controls and avoid exposing personal details.

7.5 AI-Assisted Suggestions with Human Review

If the institution later wishes to explore machine learning, a model could be evaluated as a source of **suggestions** that a person confirms, rather than as an automatic decision. Any such step would be a separate project with its own data, licensing, evaluation and approval, and it is not part of the current design.

7.6 Phased Roadmap

Table 7.1: Proposed roadmap for future work

Phase	Proposed activity	Entry condition
1	Configure and verify the two classification flows.	Instance access and confirmed inventory.
2	Refine rules based on review-path reasons.	Test evidence recorded.
3	Expand catalog mappings.	Catalog owners confirm mappings.
4	Add reporting on classified and reviewed records.	Stable classification behaviour.
5	Design support-group routing.	Classification accepted by the supervisor or owner.

8. CONCLUSION

This project has designed a controlled, rule-based approach to classifying eligible Incident records and Requested Items from existing Service Catalog items in ServiceNow using Flow Designer. The design uses documented rules, independent rule checks, an explicit decision process and a field-preservation guard so that values are written only when the evidence is clear and only into blank fields.

8.1 Project Goals and Expected Outcomes

- **Consistent classification:** the same documented rules are applied to every eligible record, and each automatic outcome is traceable to a rule ID.
- **Data integrity:** the field guard ensures that populated values are never overwritten and that only valid, compatible values can be written.
- **Safe handling of uncertainty:** unmatched, conflicting and invalid cases lead to a review path instead of a guess.
- **Scope control:** assignment, approval, rejection, fulfilment and closure remain with the processes that handle them today.
- **Verifiability:** a planned set of eighteen test cases, traced to the functional requirements, is ready to be executed once the flows are configured.

8.2 Reflection on Project Learnings

Preparing this design highlighted that much of the difficulty in workflow automation lies not in configuring actions but in deciding what the automation must **not** do. Recognising that attaching a new flow to an existing catalog item could replace its fulfilment process, and that a first-match rule ladder could hide conflicts, changed the design significantly. The project also reinforced the importance of separating verified facts from assumptions: field names, choice values and trigger options have been kept as placeholders until they can be confirmed.

8.3 Final Thoughts

The next step is to obtain instance access, complete the discovery phase, configure the flows incrementally and execute the planned tests. Actual results and genuine evidence will then replace the expectations described in this report. Once validated, the classification flows can provide a reliable foundation for later enhancements such as support-group routing and reporting, while preserving the existing service management processes that the organisation already depends on.

REFERENCES

The following official ServiceNow documentation pages were consulted for the design. ServiceNow documentation is versioned by release, so details should be checked against the release of the instance used for implementation.

1. ServiceNow, "Exploring flows," ServiceNow Product Documentation (Workflow Studio). <https://www.servicenow.com/docs/r/build-workflows/workflow-studio/exploring-flows.html>
2. ServiceNow, "Workflow Studio flow trigger types," ServiceNow Product Documentation. <https://www.servicenow.com/docs/r/build-workflows/workflow-studio/flow-triggers.html>
3. ServiceNow, "Update Record action," ServiceNow Product Documentation (Workflow Studio core actions). <https://www.servicenow.com/docs/r/OapmY4GUviEuI2zkQMVfbQ/xphsBUNADMxgBexhir9U2Q>
4. ServiceNow, "Create a flow with a Service Catalog trigger," ServiceNow Product Documentation. <https://www.servicenow.com/docs/r/build-workflows/workflow-studio/create-sc-flow.html>
5. ServiceNow, "Incident Management," ServiceNow Product Documentation (IT Service Management). https://www.servicenow.com/docs/r/it-service-management/incident-management/c_IncidentManagement.html
6. ServiceNow, "Exploring Incident Management," ServiceNow Product Documentation (Zurich, IT Service Management). <https://servicenow.com/docs/bundle/zurich-it-service-management/page/product/incident-management/concept/incident-management-process.html>
7. ServiceNow, "Get Catalog Variables action," ServiceNow Product Documentation (Zurich, Build Workflows). <https://www.servicenow.com/docs/bundle/zurich-build-workflows/page/administer/flow-designer/reference/get-cat-variables-flow-designer.html>
8. ServiceNow Developer Program, "Flow Designer and Catalog – New in Paris," ServiceNow Developer Blog. <https://developer.servicenow.com/blog.do?p=%2Fpost%2Fparis-flow-designer-catalog%2F>

APPENDIX A: TEST RECORD AND EVIDENCE TEMPLATES

Table A.1: Test execution record (one per test case)

Field	Entry
Test ID	TC-__
Date and instance	To be completed during execution
Record type and number	Incident / Requested Item; number of the test record
Input conditions	Short description, existing field values, catalog item and variables
Expected result	Rule ID and expected field values, copied from Table 4.2
Observed result	To be completed only after running the test
Pass / Fail	To be completed only after comparing expected and observed results
Execution evidence	Flow execution reference and screenshot file name
Notes / defect	Description of any difference and the action taken

Table A.2: Screenshot evidence plan

Ref.	Evidence to capture after configuration	Supports
S-01	Incident flow trigger and its conditions	FR-01
S-02	Incident rule checks and precedence step	FR-03, FR-05
S-03	Decision process and review path	FR-06, FR-10
S-04	Field-guard logic and the single Update Record action	FR-07, FR-08
S-05	Request flow trigger and mapping check	FR-02
S-06	Get Catalog Variables configuration, if used	FR-04
S-07	Representative record before and after classification	TC-01, TC-12
S-08	Execution details for conflict, preservation and repeat tests	TC-08, TC-11, TC-17

Glossary of terms used in this report

Table A.3: Glossary

Term	Meaning in this report
Classification	The category and subcategory (or equivalent field) that describe what a record is about.
Requested Item	The record created for one catalog item ordered through the Service Catalog.
Trigger record	The record that caused a flow to start and whose data the flow uses.
Rule register	The documented list of rules, mappings, precedence, exclusions and linked tests.
Review path	The confirmed mechanism through which a person classifies records the flow did not.
Field guard	The logic that writes only blank, compatible fields and preserves populated ones.

APPENDIX B: SERVICENOW CERTIFICATE